



Exercise Problem 5 (EP_05)

Subject: *ENE/EIE 334 Digital System Engineering*

Series : *Computation Series*

Program: *Bachelor of Engineering in ECEE¹, Bachelor of Engineering in EIE²*

Semester: 1/2026

Topic: *Arithmetic and Logic Unit (ALU): Arith Module: Binary Adder (binadder)*

Designer: *Pinit Kumhom*

Revision History:

Last Updated: September, 2026 for Semester 1/2026

First Released: September, 2026 for Semester 1/2026

Objectives

1. To learn about tradeoff in designing bitwise boolean module for Beta ALU targeting CMOS technology
2. To practice using simulation tool particularly JSIM in verification and evaluation of binary adder modules.

Name-Surname (ชื่อ-สกุล) ชลธิชา อารีวงษ์

Student ID (รหัสประจำ นศ.) 67070902472

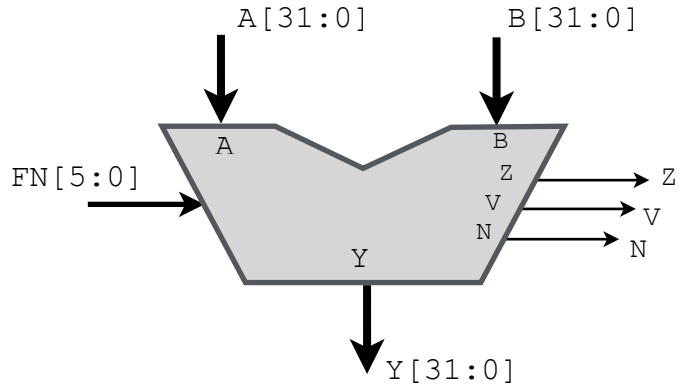
¹ **ECEE** stands for **Electrical Communication and Electronic Engineering**, a curriculum's name in the department of electronic and telecommunication, faculty of engineering, KMUTT

² **EIE** stands for **Electronic and Infocommunication Engineering (EIE)**, a curriculum's name in the department of electronic and telecommunication, faculty of engineering, KMUTT

ALU Module for Beta CPU

Interface Specification

Port Name	Dir	Description
A[31:0]	In	32-bit operand 1
B[31:0]	In	32-bit operand 2
Y[31:0]	Out	32-bit result
FN[5:0]	In	6-bit function selection
Z	Out	zero flag (1 bit)
V	Out	overflow flag (1 bit)
N	Out	negative flag (1 bit)



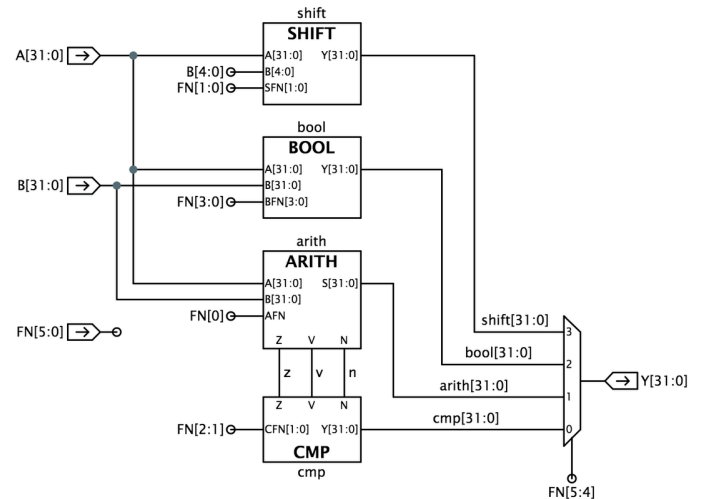
Functional Specification

FN[5:0]	Operation	Output value Y[31:0]
00-011	CMPEQ	$Y = (A == B)$
00-101	CMPLT	$Y = (A < B)$
00-111	CMPLT	$Y = (A \leq B)$
01---0	32-bit ADD	$Y = A + B$
01---1	32-bit SUBTRACT	$Y = A - B$
10abcd	Bit-wise Boolean	$Y[i] = F_{abcd}(A[i], B[i])$
11--00	Logical Shift left (SHL)	$Y = A \ll B$
11--01	Logical Shift right (SHR)	$Y = A \gg B$
11--11	Arithmetic Shift right (SRA)	$Y = A \gg B$ (sign extended)

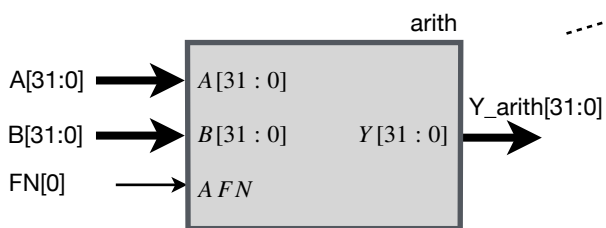
```

Z <= '1' when Y=0 else
    '0';
N <= '1' when Y<0 else
    '0';
V <= '1' when Y<(-2^31) or Y>(2^31-1) else
    '0';
    
```

Structural (Architectural) Design

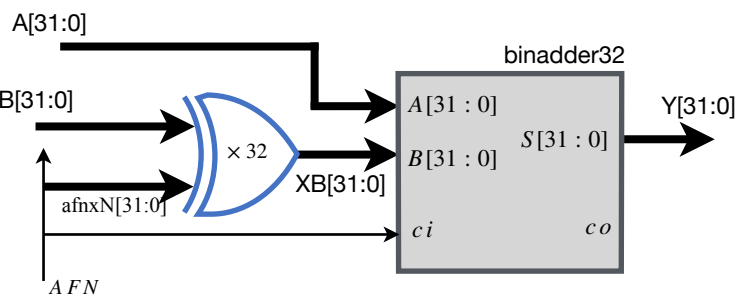


ARITH Module's Specification



$Y \leq A - B$ when $afn = 1$ else $A + B$

ARITH module Design



Modeling the binadder32 module Using JSIM

Design Concept#1: Ripple-Carry Adder (RCA)

** Ripple Carry Adders (RCA)

* Generic Constant (Parameter): width -- number of bits of the data

*-----
* Module (sub-circuit) Name: RCA#width (RCA followed by the width number)

*-----
* Input Ports

*-----
* A[width-1:0] -- operand 1

* B[width-1:0] -- operand 2

* ci -- carry in to LSB

*-----
* Output ports

*-----
* co -- carry out from the MSB

* S[width-1:0] -- sum

*-----

.subckt rca32 A[31:0] B[31:0] ci co S[31:0]

Xl A[15:0] B[15:0] ci C16 S[15:0] rca16

Xh A[31:16] B[31:16] C16 co S[31:16] rca16

.ends

.subckt rca30 A[29:0] B[29:0] ci co S[29:0]

Xl A[15:0] B[15:0] ci C16 S[15:0] rca16

Xh8 A[23:16] B[23:16] C16 C24 S[23:16] rca8

Xh4 A[27:24] B[27:24] C24 C28 S[27:24] rca4

Xh2 A[29:28] B[29:28] C28 co S[29:28] rca2

.ends

.subckt rca24 A[23:0] B[23:0] ci co S[23:0]

Xl A[15:0] B[15:0] ci C16 S[15:0] rca16

Xh A[23:16] B[23:16] C16 co S[23:16] rca8

.ends

.subckt rca22 A[21:0] B[21:0] ci co S[21:0]

Xl A[15:0] B[15:0] ci C16 S[15:0] rca16

Xh4 A[19:16] B[19:16] C16 C20 S[19:16] rca4

Xh2 A[21:20] B[21:20] C20 co S[21:20] rca2

.ends

.subckt rca16 A[15:0] B[15:0] ci co S[15:0]

Xl A[7:0] B[7:0] ci C8 S[7:0] rca8

Xh A[15:8] B[15:8] C8 co S[15:8] rca8

.ends

.subckt rca8 A[7:0] B[7:0] ci co S[7:0]

Xl A[3:0] B[3:0] ci C4 S[3:0] rca4

Xh A[7:4] B[7:4] C4 co S[7:4] rca4

.ends

.subckt rca4 A[3:0] B[3:0] ci co S[3:0]

Xl A[1:0] B[1:0] ci C2 S[1:0] rca2

Xh A[3:2] B[3:2] C2 co S[3:2] rca2

.ends

.subckt rca2 A[1:0] B[1:0] ci co S[1:0]

Xl A[0] B[0] ci C1 S[0] fa

Xh A[1] B[1] C1 co S[1] fa

.ends

.subckt fa a b ci co s

Xp1 a b p1 nand2

Xp2 s1 ci p2 nand2

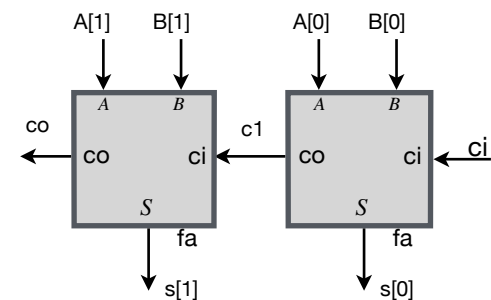
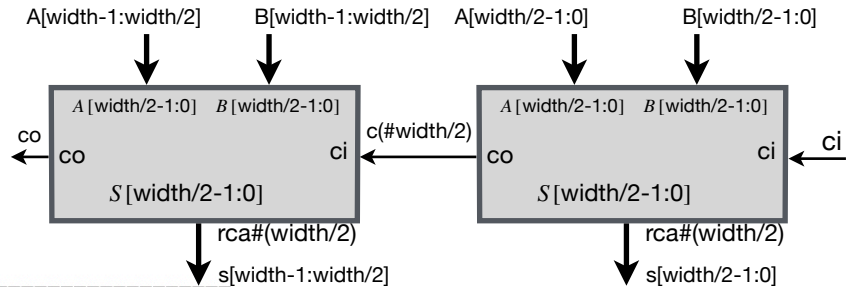
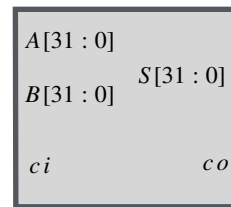
Xco p1 p2 co nand2

Xs1 a b s1 xor2

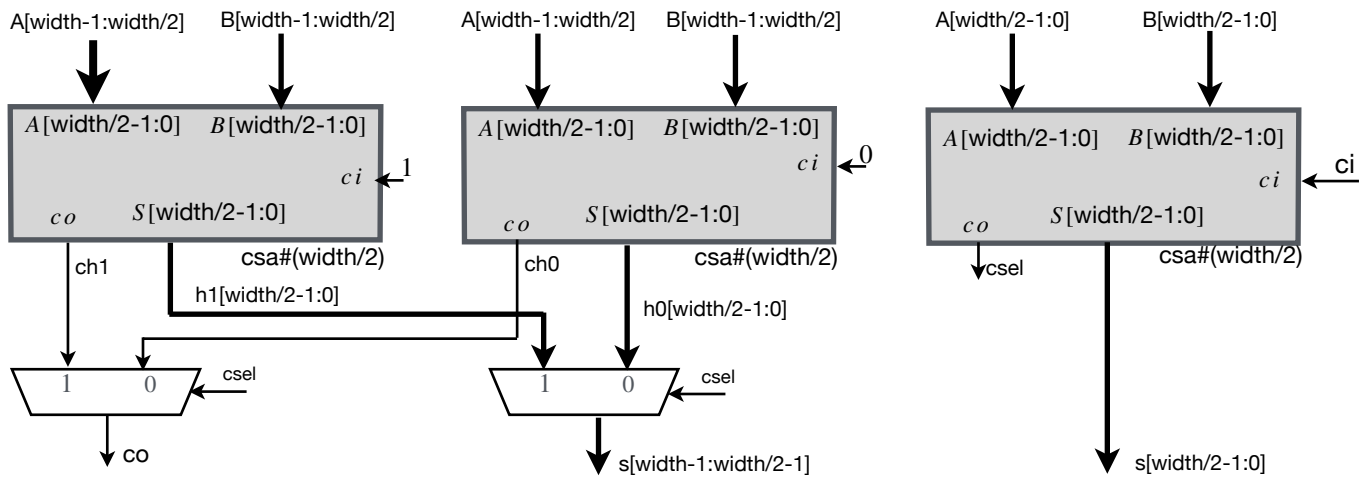
Xs s1 ci s xor2

.ends

binadder32



Design Concept#2: Carry-Select Adder (CSA)



** Carray Select Adder (CSA)

* Generic Constant (Parameter): width -- number of bits of the data

* (width=2,4,8,16,32)

* Module (sub-circuit) Name: CSA#width (CSA followed by the width number)

* Input Ports

* A[width-1:0] -- operand 1

* B[width-1:0] -- operand 2

* ci -- carry in to LSB

* Output ports

* co -- carry out from the MSB

* S[width-1:0] -- sum

.subckt csa32 A[31:0] B[31:0] ci co S[31:0]

Xl A[15:0] B[15:0] ci C16 S[15:0] csa16

Xh0 A[31:16] B[31:16] 0 ch0 h0[15:0] csa16

Xh1 A[31:16] B[31:16] vdd ch1 h1[15:0] csa16

Xh C16#16 h1[15:0] h0[15:0] S[31:16] mux2

Xh C16 ch1 cho co mux2

.ends

.subckt csa16 A[15:0] B[15:0] ci co S[15:0]

Xl A[7:0] B[7:0] ci C8 S[7:0] csa8

Xh0 A[15:8] B[15:8] 0 ch0 h0[7:0] csa8

Xh1 A[15:8] B[15:8] vdd ch1 h1[7:0] csa8

Xh C8#8 h1[7:0] h0[7:0] S[15:8] mux2

Xh C8 ch1 cho co mux2

.ends

.subckt csa8 A[7:0] B[7:0] ci co S[7:0]

Xl A[3:0] B[3:0] ci C4 S[3:0] csa4

Xh0 A[7:4] B[7:4] 0 ch0 h0[3:0] csa4

Xh1 A[7:4] B[7:4] vdd ch1 h1[3:0] csa4

Xh C4#4 h1[3:0] h0[3:0] S[7:4] mux2

Xh C4 ch1 cho co mux2

.ends

.subckt csa4 A[3:0] B[3:0] ci co S[3:0]

Xl A[1:0] B[1:0] ci C2 S[1:0] csa2

Xh0 A[3:2] B[3:2] 0 ch0 h0[1:0] csa2

Xh1 A[3:2] B[3:2] vdd ch1 h1[1:0] csa2

Xh C2#2 h1[1:0] h0[1:0] S[3:2] mux2

Xh C2 ch1 cho co mux2

.ends

.subckt csa2 A[1:0] B[1:0] ci co S[1:0]

Xl A[0] B[0] ci C1 S[0] fa

Xh0 A[1] B[1] 0 ch0 h0[0] fa

Xh1 A[1] B[1] vdd ch1 h1[0] fa

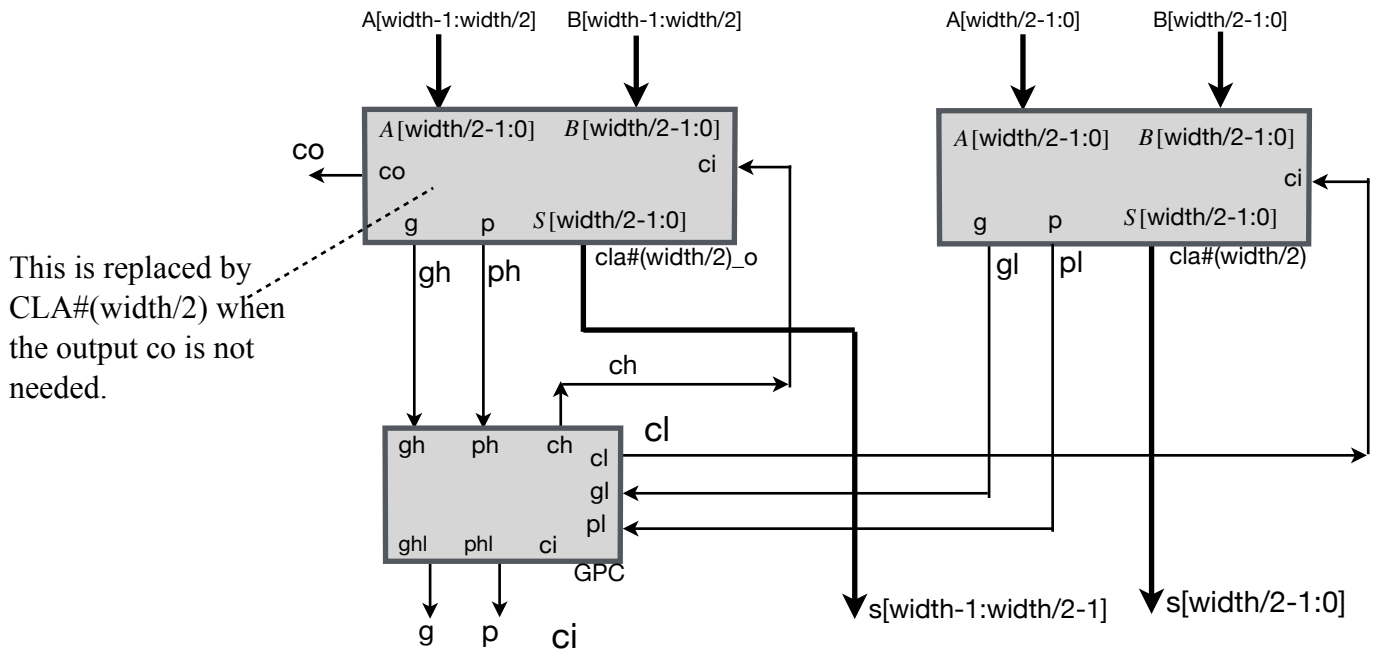
Xh C1 h1[0] h0[0] S[1] mux2

Xh C1 ch1 cho co mux2

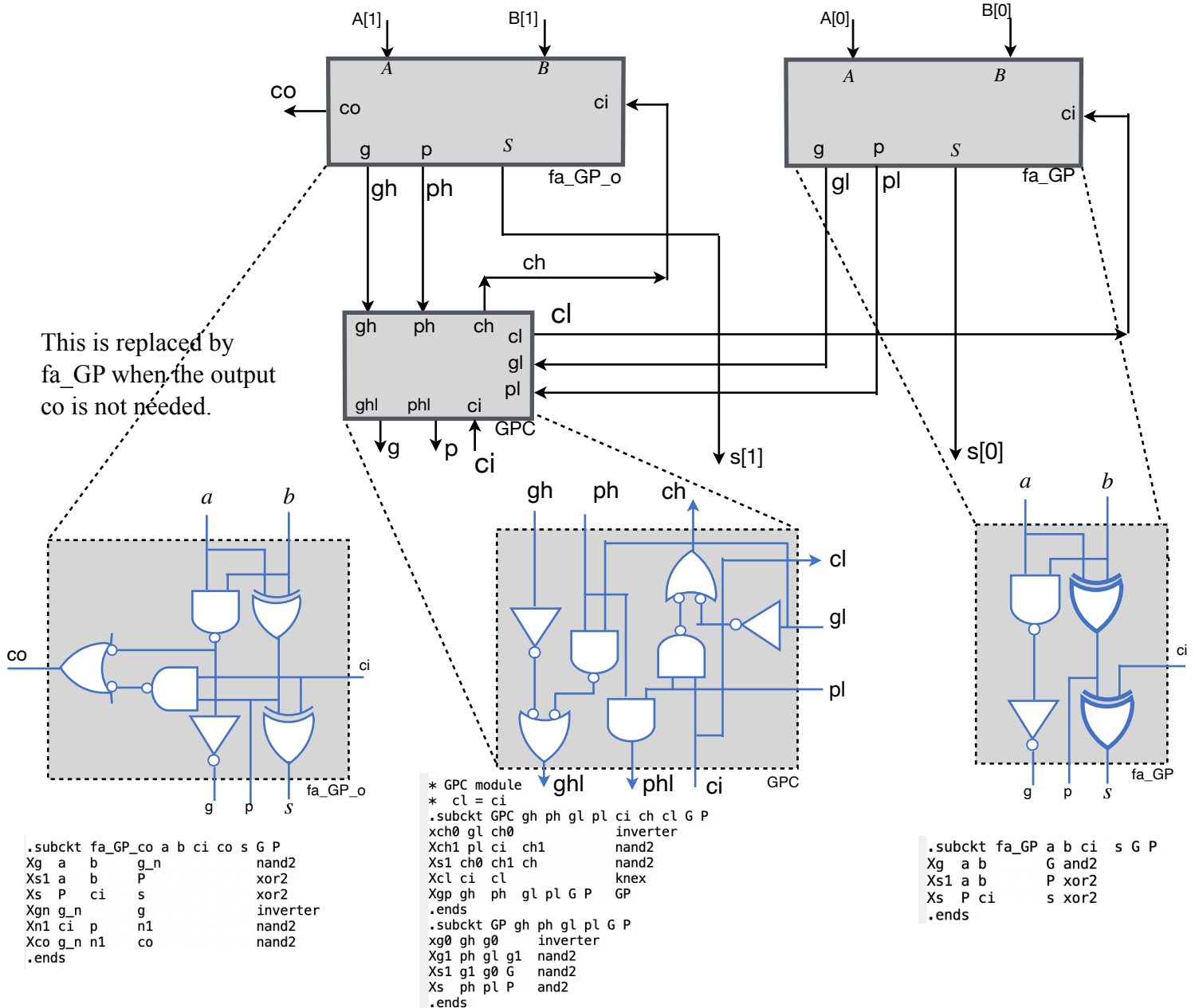
.ends

Design Concept#3: Carry-Lookahead Adder (CLA)

Generic structure of CLA#width composed of 2 CLA#(width/2) and GPC module



Structure of CLA2_o composing of fa_GP (=CLA1), fa_GP_co(=CLA1_o) and GPC module



```

*****
*** Carry Look Ahead Adder (CLA)
*****
* Generic Constant (Parameter): width -- number of bits of the data
*   (width=2,4,8,16,32)
*-----
* Module (sub-circuit) Name: CLA#width (CLA followed by the width-1 number), CLA#width_co
*-----
* Input Ports
*-----
* A[width-1:0] -- operand 1
* B[width-1:0] -- operand 2
* ci           -- carry in
*-----
* Output ports
*-----
* S[width-1:0] -- sum
* G           -- the generated carry from the root GPC
*             -- for connecting as G of GPC when combining the CLAwidth
* P           -- the carry propagation control of the root GPC
*             -- for connecting as cin of GPC when combining the CLAwidth
* co (only CLA#width_co) -- carry out from the MSB which is Cwidth
*             -- this co is ignored when the module is used in signed adder
*-----
* width=32 (CLA32_o, CLA32)
* cl=ci
.subckt cla32_co A[31:0] B[31:0] ci co S[31:0] G P
Xl  A[ 15:0 ] B[ 15:0 ] cl      S[ 15:0 ] gl pl      cla16
Xh  A[ 31:16 ] B[ 31:16 ] ch    co    S[ 31:16 ] gh ph    cla16_co
Xgpc gh      ph      gl    pl    ci    ch    cl    G    P    gpc
.ends
.subckt cla32 A[31:0] B[31:0] ci S[31:0] g p
Xl  A[ 15:0 ] B[ 15:0 ] cl      S[ 15:0 ] gl pl      cla16
Xh  A[ 31:16 ] B[ 31:16 ] ch    S[ 31:16 ] gh ph    cla16
Xgpc gh      ph      gl    pl    ci    ch    cl    G    P    gpc
.ends

* width=16 (CLA16_o, CLA16)
* cl=ci
.subckt cla16_co A[15:0] B[15:0] ci co S[15:0] G P
Xl  A[ 7:0 ] B[ 7:0 ] cl      S[ 7:0 ] gl pl      cla8
Xh  A[ 15:9 ] B[ 15:9 ] ch    co    S[ 15:9 ] gh ph    cla8_co
Xgpc gh      ph      gl    pl    ci    ch    cl    G    P    gpc
.ends
.subckt cla16 A[15:0] B[15:0] ci S[15:0] G P
Xl  A[ 7:0 ] B[ 7:0 ] cl      S[ 7:0 ] gl pl      cla8
Xh  A[ 15:9 ] B[ 15:9 ] ch    S[ 15:9 ] gh ph    cla8_co
Xgpc gh      ph      gl    pl    ci    ch    cl    G    P    gpc
.ends

* width=8 (CLA8_co, CLA8)
* cl = ci
.subckt cla8_co A[7:0] B[7:0] ci co S[7:0] G P
Xl  A[ 3:0 ] B[ 3:0 ] cl      S[ 3:0 ] gl pl      cla4_co
Xh  A[ 7:4 ] B[ 7:4 ] ch    co    S[ 7:4 ] gh ph    cla4_co
Xgpc gh      ph      gl    pl    ci    ch    cl    G    P    gpc
.ends
.subckt cla8 A[7:0] B[7:0] ci S[7:0] G P
Xl  A[ 3:0 ] B[ 3:0 ] cl      S[ 3:0 ] gl pl      cla4
Xh  A[ 7:4 ] B[ 7:4 ] ch    S[ 7:4 ] gh ph    cla4_co
Xgpc gh      ph      gl    pl    ci    ch    cl    G    P    gpc
.ends

* width=4 (CLA4_co, CLA4)
* cl=ci
.subckt cla4_co A[3:0] B[3:0] ci co S[3:0] G P
Xl  A[ 1:0 ] B[ 1:0 ] cl      S[ 1:0 ] gl pl      cla2_co
Xh  A[ 3:2 ] B[ 3:2 ] ch    co    S[ 3:2 ] gh ph    cla2_co
Xgpc gh      ph      gl    pl    ci    ch    cl    G    P    gpc
.ends
.subckt cla4 A[3:0] B[3:0] ci S[3:0] G P
Xl  A[ 1:0 ] B[ 1:0 ] cl      S[ 1:0 ] gl pl      cla2
Xh  A[ 3:2 ] B[ 3:2 ] ch    co    S[ 3:2 ] gh ph    cla2
Xgpc gh      ph      gl    pl    ci    ch    cl    G    P    gpc
.ends

* width=2 (CLA2_co, CLA2)
* cl=ci
.subckt cla2_co A[1:0] B[1:0] ci co S[1:0] G P
Xl  A[ 0 ] B[ 0 ] cl      S[ 0 ] gl pl      fa_GP
Xh  A[ 1 ] B[ 1 ] ch    co    S[ 1 ] gh ph    fa_GP_co
Xgpc gh      ph      gl    pl    ci    ch    cl    G    P    gpc
.ends
.subckt cla2 A[1:0] B[1:0] ci S[1:0] G P
Xl  A[ 0 ] B[ 0 ] cl      S[ 0 ] gl pl      fa_GP
Xh  A[ 1 ] B[ 1 ] ch    S[ 1 ] gh ph    fa_GP
Xgpc gh      ph      gl    pl    ci    ch    cl    G    P    gpc
.ends

```

Verification of Binary Adders by Exhaustive Test (width=4)

```
// including all necessary modules (be careful of multiple inclusion)
#include "binadder.jsim"
```

```
// checkoff header
.checkoff "" "Exhaustive test of 4-bit binary adder module" 0
```

- * 1. Instantiate the module for testing
- * make sure the ports are in right order
- * Uncomment to test respective binary adder

```
Xrca4 a[3:0] b[3:0] ci co s[3:0] rca4 // for testing RC4
// Xcsa4 a[3:0] b[3:0] ci co s[3:0] csa4 // for testing CSA4
// Xcla4 a[3:0] b[3:0] ci co s[3:0] g h cla4_co // for testing CLA4_co
```

```
* 2. Veirfication of outputs
* Verifying output s[3:0]
.verify s[3:0] periodic(9.9E-8,1.0000000000000001E-7)
```

+	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
+	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	0
+	2	3	4	5	6	7	8	9	10	11	12	13	14	15	0	1
+	3	4	5	6	7	8	9	10	11	12	13	14	15	0	1	2
+	4	5	6	7	8	9	10	11	12	13	14	15	0	1	2	3
+	5	6	7	8	9	10	11	12	13	14	15	0	1	2	3	4
+	6	7	8	9	10	11	12	13	14	15	0	1	2	3	4	5
+	7	8	9	10	11	12	13	14	15	0	1	2	3	4	5	6
+	8	9	10	11	12	13	14	15	0	1	2	3	4	5	6	7
+	9	10	11	12	13	14	15	0	1	2	3	4	5	6	7	8
+	10	11	12	13	14	15	0	1	2	3	4	5	6	7	8	9
+	11	12	13	14	15	0	1	2	3	4	5	6	7	8	9	10
+	12	13	14	15	0	1	2	3	4	5	6	7	8	9	10	11
+	13	14	15	0	1	2	3	4	5	6	7	8	9	10	11	12
+	14	15	0	1	2	3	4	5	6	7	8	9	10	11	12	13
+	15	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
+	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	0
+	2	3	4	5	6	7	8	9	10	11	12	13	14	15	0	1
+	3	4	5	6	7	8	9	10	11	12	13	14	15	0	1	2
+	4	5	6	7	8	9	10	11	12	13	14	15	0	1	2	3
+	5	6	7	8	9	10	11	12	13	14	15	0	1	2	3	4
+	6	7	8	9	10	11	12	13	14	15	0	1	2	3	4	5
+	7	8	9	10	11	12	13	14	15	0	1	2	3	4	5	6
+	8	9	10	11	12	13	14	15	0	1	2	3	4	5	6	7
+	9	10	11	12	13	14	15	0	1	2	3	4	5	6	7	8
+	10	11	12	13	14	15	0	1	2	3	4	5	6	7	8	9
+	11	12	13	14	15	0	1	2	3	4	5	6	7	8	9	10
+	12	13	14	15	0	1	2	3	4	5	6	7	8	9	10	11
+	13	14	15	0	1	2	3	4	5	6	7	8	9	10	11	12
+	14	15	0	1	2	3	4	5	6	7	8	9	10	11	12	13
+	15	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
+	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

```
* Verifying output co
.verify co periodic(9.9E-8,1.0000000000000001E-7)
```

[illegible]

```
* 3. Generate sample 4-bit inputs for A and B and ci
```

[illegible]

```
Wb b[3:0] nrz(0,3.3,1600n,0n,.1n,.1n)
```

[illegible]

Wc ci nrz(0,3.3,25600n,0n,.1n,.1n)

$$+ 0.1$$

```
* 4. simulate the circuit with enough time
.tran 51200n
```

```
* 5. Some useful plots...
* using the appropriate .plot command
.plot L[a{3:0}]
.plot L[b{3:0}]
.plot ci
.plot L[s{3:0}]
.plot co
```

Verification of Binary Adders by Designing a Set of Test Vectors (width=32)

```
// including all necessary modules (be careful of multiple inclusion)
.include "binadder.jsim"

// checkoff header
.checkoff "" "Testing of 32-bit binary adder module" 0

* 1. Instantiate the module for testing
* make sure the ports are in right order
* Uncomment to test respective binary adder

// Xrca32 a[31:0] b[31:0] ci co s[31:0] rca32 // for testing RC4
Xcsa32 a[31:0] b[31:0] ci co s[31:0] csa32 // for testing CSA4
// Xcla32 a[31:0] b[31:0] ci co s[31:0] g h cla32_co // for testing CLA4_co

* 2. Verification of outputs
* Verifying output s[31:0]
.verify s[31:0] periodic(9.9E-8,1.0000000000000001E-7)
+ 0x00000000 // 99ns
+ 0x55555555 // 199ns
+ 0x55555555 // 299ns
+ 0xAAAAAAAA // 399ns
+ 0xAAAAAAAA // 499ns
+ 0xAAAAAAAA // 599ns
+ 0x55555554 // 699ns
+ 0x00000000 // 799ns
+ 0x00000000 // 899ns
+ 0xFFFFFFFF // 999ns
+ 0x00000001 // 1099ns
+ 0x00000002 // 1199ns
+ 0x00000004 // 1299ns
+ 0x00000008 // 1399ns
+ 0x00000010 // 1499ns
+ 0x00000020 // 1599ns
+ 0x00000040 // 1699ns
+ 0x00000080 // 1799ns
+ 0x00000100 // 1899ns
+ 0x00000200 // 1999ns
+ 0x00000400 // 2099ns
+ 0x00000800 // 2199ns
+ 0x00001000 // 2299ns
+ 0x00002000 // 2399ns
+ 0x00004000 // 2499ns
+ 0x00008000 // 2599ns
+ 0x00010000 // 2699ns
+ 0x00020000 // 2799ns
+ 0x00040000 // 2899ns
+ 0x00080000 // 2999ns
+ 0x00100000 // 3099ns
+ 0x00200000 // 3199ns
+ 0x00400000 // 3298ns
+ 0x00800000 // 3398ns
+ 0x01000000 // 3498ns
+ 0x02000000 // 3598ns
+ 0x04000000 // 3698ns
+ 0x08000000 // 3798ns
+ 0x10000000 // 3898ns
+ 0x20000000 // 3998ns
+ 0x40000000 // 4098ns
+ 0x80000000 // 4198ns
+ 0x00000005 // 4298ns
+ 0x00000001 // 4398ns
+ 0x7FFFFFFF // 4498ns
+ 0x6DCBA988 // 4598ns
+ 0xFFFFFFFF // 4698ns
+ 0x80000001 // 4798ns
+ 0x00000000 // 4898ns

// 3. Setting the input
// 3.1 Operand 1: a[31:0]
Wa a[31:0] nrz(0,3.3,100n,0n,.1n,.1n)
// test all combinations of 3 inputs to each bit of the adder.
//
+ 0x00000000 0x55555555 0x00000000 0x55555555 0xAAAAAAAA 0x00000000 0xAAAAAAAA
+ 0xFFFFFFFF 0x00000001 0xFFFFFFFF
// test each input to Z logic
+ 0x00000001 0xFFFFFFFF2 0x00000001 0xAAAAAAAA 0xFFFFFFFF 0x00000002 0x00000000 0x00000007F
+ 0x00000080 0x00000180 0x00000380 0x00000780 0x00001000 0x00001000 0x00003000 0x00007000
+ 0x0000F000 0x00001000 0x00001000 0x0007F800 0x000FFC00 0x001FFE00 0x003FFF00 0x00000080
+ 0xFF000000 0x04000000 0x030000FF 0x070007FF 0x0F0003FF 0x1F0001FF 0x3F0000FF 0x80000001
// comparing 2 numbers by subtraction
+ 0x00000002 0x80000000 0x80000000 0x80000000 0x00000003 0x7FFFFFFF 0x00000003

// 3.2 Operand 2: b[31:0]
Wb b[31:0] nrz(0,3.3,100n,0n,.1n,.1n)
// test all combinations of 3 inputs to each bit of the adder.
//
+ 0x00000000 0x00000000 0x55555555 0x55555555 0x00000000 0xAAAAAAAA 0xAAAAAAAA
+ 0x00000000 0xFFFFFFFF 0xFFFFFFFF
// test each input to Z logic
+ 0x00000000 0x0000000F 0x00000003 0x5555555C 0x00000010 0x0000001E 0x0000003F 0x00000000
+ 0x00000080 0x00000080 0x00000080 0x00000080 0x00000000 0x00001000 0x00001000 0x00001000
+ 0x00001000 0x0001EFFF 0x0003EFFF 0x00000800 0x00000400 0x00000200 0x00000100 0x007FFF7F
+ 0x02000000 0xFE000000 0x00FFF001 0x00FFF801 0x00FFFC01 0x00FFFE01 0x00FFFF01 0xFFFFFFFF
// comparing 2 numbers by subtraction
+ 0x00000002 0x80000000 0xFFFFFFFF 0xEDCBA987 0xFFFFFFFF 0x00000001 0xFFFFFFFF

// 3.3 Operand 3: carry in (c0)
Wc ci nrz(0,3.3,100n,0n,.1n,.1n)
+ 0 0 0 0 0 0 0
+ 1 0 1
+ 0 1 0 0 1 0 1 1
+ 0 0 0 0 0 0 0 0
+ 0 1 1 0 0 0 0 1
+ 0 0 0 0 0 0 0 1
+ 1 1 1 1 1 1 1

// 4. Run the simulation for all test patterns
.tran 4900n

// 5. Plot the waveforms
.plot L(a[31:0])
.plot L(b[31:0])
.plot ci
.plot L(s[31:0])
.plot co

// 2.2 the carry out
.verify co periodic(99.9n,100n)
+ 0 0 0 0 0 0 1 1 1
+ 0 1 0 1 1 0 0 0
+ 0 0 0 0 0 0 0 0
+ 0 0 0 0 0 0 0 0
+ 1 1 0 0 0 0 0 1
+ 0 1 1 1 0 0 1
```

1. Complete all JSIM models of binary adders of different widths above. Save the file.
2. Use the provided checkoff (testbench) files for verifying your JSIM models of the binary adders. Make corrections as necessary. Record your works for report — this is an individual work
3. Record the size of rca32, csa32, and cla32_o in terms of # of gates and area in μm^2
4. Use the timing analysis tool that comes with JSIM to find t_{PD} of rca32, csa32, and cla32
5. Make a comparison of these 3 design concepts of binary adder based on the testing results of 32-bit adders. Are these data agreed with the expected results (see lecture8: design tradeoff)?

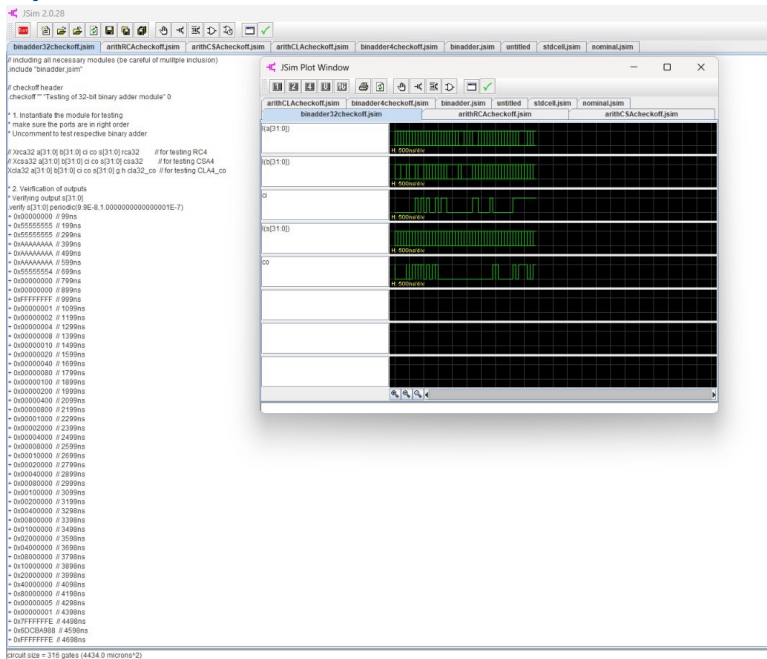
Circuit size: 160 gates ($2.633.0 \mu m^2$)

[illegible]

circuit size: 1547 gates (29376.0 μm^2)

circuit size: 1547 gates (29376.0 μm^2)

๒๑๑8 c19 ๒2



tp : 1.645ns

circuit size: 316gates (4939.0μm²)

อภิปรายและสรุปผล

เมื่อทำข้อ3 Designตามเปรียบเทียบกับ จะเห็น ความสัมพันธ์ ระหว่างพื้นที่ และ ความเร็ว ได้อย่างชัดเจน โดย CLA จะใช้พื้นที่น้อยสุด ประสิทธิภาพการประมวลผลสูง แต่จะทำงานช้าสุด

เพราะต้องรอสัญญาณจาก (carry) รั้วผ่านทีละเกต

C&A จะทำงานได้เร็วขึ้นมาก แต่ต้องแลกกับการใช้พื้นที่เยอะสุด เพราะหลักการคือสร้างวงจรบวกซ้ำซ้อน เพื่อรอเสื่อกค่าตอบ

ส่วน CLA เป็นตัวเลือที่ดีที่สุดในแง่ความเร็ว และใช้พื้นที่เพิ่มขึ้นจาก CLA เพียงเล็กน้อย เป็น ms ออกแบบที่ให้ความสมเหตุ ะหว่างพื้นที่ และ ความเร็วที่ดีที่สุด

CLAจึงเป็นตัวเลือกที่ ่ดีประ สิกภาพสูง สด